AJ SMART BIZ · DOCUMENT 04

System Architecture & Design

High-level and low-level design: components, multi-tenancy, security, data flows, state machines and key design decisions.

Audience	Architects, developers, DevOps, technical reviewers
Version / date	1.0 · 7 November 2026
Related	05 Database Design · 06 API Reference · 07 Deployment · 08 Security

Contents

1. Architecture overview
2. Technology stack
3. Component design — Backend API
4. Component design — Angular consoles
5. Component design — Next.js websites
6. Multi-tenancy
7. Authentication & authorisation
8. Feature entitlement
9. Key flows (sequence)
10. State machines
11. Cross-cutting concerns
12. Architecture decisions (ADR summary)

1. Architecture overview

Aj Smart Biz is a **shared-database, shared-schema multi-tenant** system. One Express API owns all data and business rules. Four kinds of client consume it, each through its own route module with its own guard.

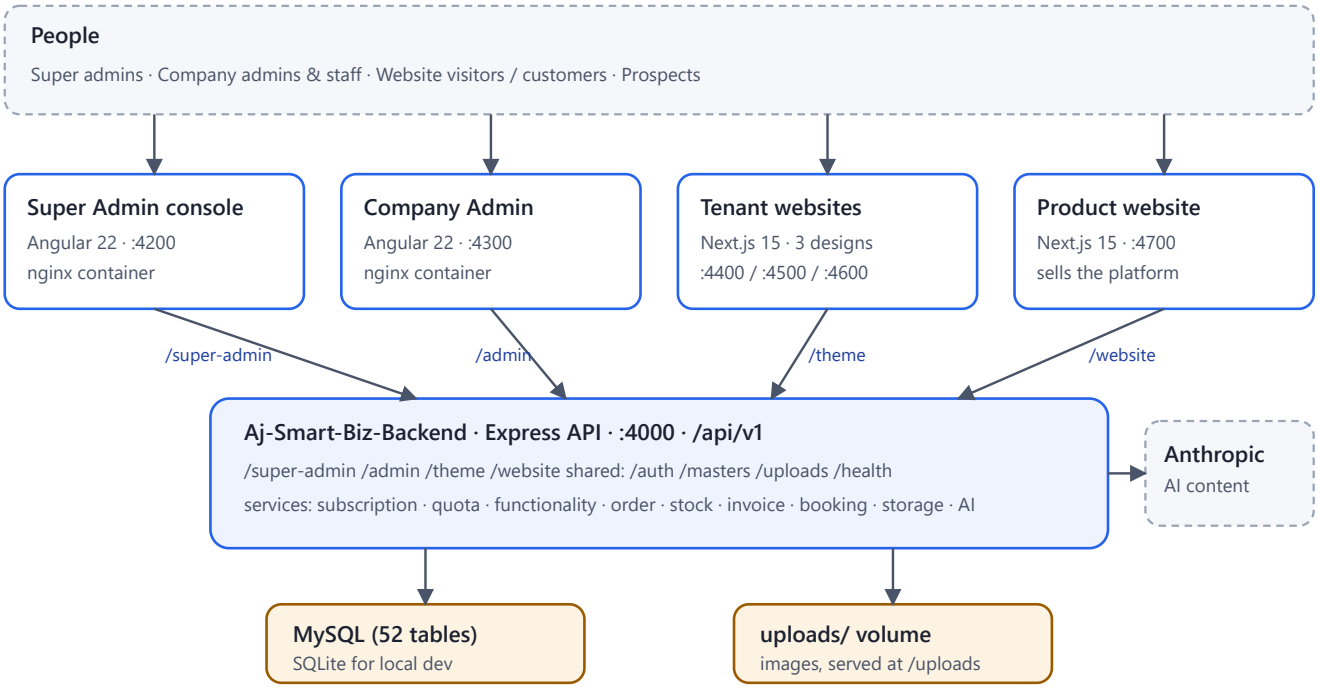


Figure 1 — System context and containers

> **Rule: the guard belongs to the module, not to the route.**
> Each consumer has one composed router (`src/modules/*`). Its scope guard is applied once when it is mounted, so a route added inside a module cannot accidentally be left unprotected.

2. Technology stack

Layer	Technology	Notes
API	Node.js 24, Express 4.19, CommonJS	`'use strict'` in every file; no TypeScript.
ORM / DB	Sequelize 6.37, MySQL (mysql2) · SQLite (dev)	`underscored`, `freezeTableName`, `paranoid` globally; sync with alter — no migrations.
Validation	Joi 17	Schemas double as Swagger request bodies (joi-to-swagger).
Auth	jsonwebtoken, bcryptjs	Access + refresh tokens; scopes super_admin / admin / customer.
Hardening	helmet, cors, express-rate-limit, compression, morgan	

Layer	Technology	Notes
Uploads	multer 2 (disk)	Storage ledger `company_files`.
API docs	swagger-jsdoc, swagger-ui-express	Auto-scanned routes + `@openapi` prose at `/api/v1/docs`.
AI	@anthropic-ai/sdk	Background job drafting website content.
Consoles	Angular 22 (standalone, signals, zoneless default), RxJS 7.8, TypeScript 6	No UI/chart/state library; own design system in `styles.scss`.
Websites	Next.js 15 App Router, React 19, TypeScript 5.8	Plain CSS + CSS Modules; standalone output.
Delivery	Docker (node:24-alpine, nginx:1.29-alpine), GitHub Actions, Docker Hub, nginx, certbot, Cloudflare DNS	See Document 07.

3. Component design — Backend API

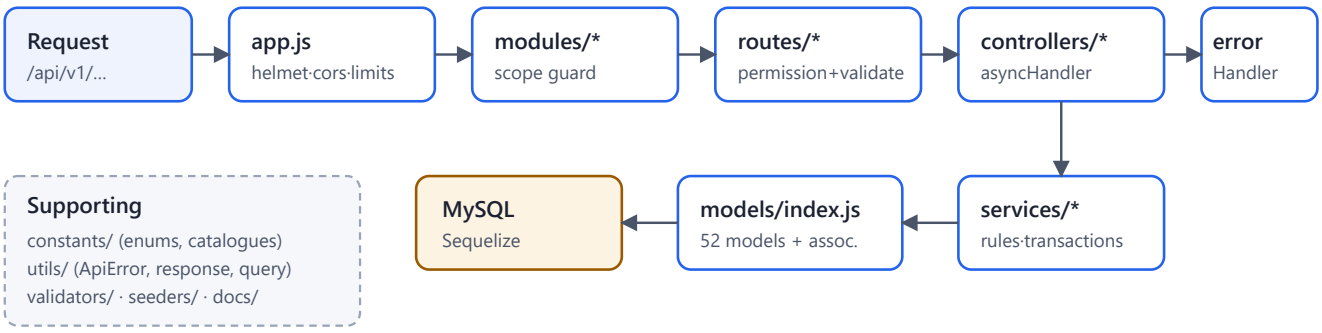


Figure 2 — Request pipeline inside the API

Folder	Responsibility
`src/server.js`	Boot: load models → create DB if missing → authenticate → sync (alter/force/none, with MySQL index clean-up and SQLite repair) → bootstrap seeders → subscription sweep (boot + hourly) → listen → storage reconcile; graceful shutdown.
`src/app.js`	Middleware chain; static `/uploads` with script-blocking CSP; login rate limit; docs and API routers; 404 and error handler.
`src/modules/`	One router per consumer: `shared`, `theme`, `website`, `admin`, `superadmin`.
`src/routes/`	Resource routers reused by modules (e.g. branches under both `/admin/company` and `/super-admin/companies/:companyId`).
`src/controllers/`	HTTP handlers; factories `crud.factory.js` (masters) and `cardResource.factory.js` (ordered tenant cards).

Folder	Responsibility
`src/services/`	Business logic: subscription (lifecycle, proration, sweeper), quota, functionality, serviceState, theme merge, order, stock, invoice, booking, analytics, storage, company (provisioning), companyDuplicate, aiContent.
`src/constants/index.js`	Single source of every enum, catalogue, transition map, default and limit.
`src/seeders/`	Idempotent boot bootstrap: root super admin, system menus, Company Admin permissions, reference data, backfills.

4. Component design — Angular consoles

Layer	Contents
`core/services`	`ApiService` (envelope unwrapping, paging), `AuthService` (signals), `CrudFactory`, `UploadService`, domain services, Toast/Confirm/Theme/ServerStatus/Branding.
`core/guards`	`authGuard`, `guestGuard`; Admin adds `permissionGuard(slug, action)` and `passwordChangeGuard`.
`core/interceptors`	Bearer token, unreachable-server detection (0/502/503/504), 401 sign-out, 403/5xx toasts, `messageOf()`.
`layout/shell`	Sidebar + top bar + outlet. Super Admin: hard-coded groups. Admin: menu tree from `/auth/me`.
`shared/`	`ListStore<T>`, `CrudPage<T>`, UI kit (modal, pager, table-state, field-error, image-upload, plan-timer, limit-notice…), `*appCan`.
`features/`	One lazy-loaded folder per business area.

Production containers serve the built SPA from nginx, with `/api/v1` and `/uploads` proxied to the API container (resolver at request time).

5. Component design — Next.js websites

Tenant themes

- `middleware.ts` resolves the domain (`?domain=` → forwarded host → `TENANT_DOMAIN` → cookie) into header `x-tenant-domain`.
- `getCompanyDetails()` calls `/theme/company-details?domain=` (no-store), memoised per render.
- Server Components by default; `'use client'` only for interactivity; `siteCompany()` strips the catalogue from client props.
- Server Actions in `app/actions/` for orders, enquiries, bookings, reviews and account; customer token in httpOnly cookie.
- `/uploads/[...path]` route proxies images; `/api/track` proxies the beacon.

Product website

- `lib/api.ts` is the only file that talks to the API.
- `getSite()` = `/website/bootstrap`, revalidated every 300 s, retried with back-off, last good response kept in memory and on disk.
- Throws only if the API has never answered → `ApiDownNotice`.
- One server action: `submit-enquiry.ts`.

6. Multi-tenancy

6.1 Isolation model

- Every tenant-owned row carries `companyId`; tenant tables are prefixed `company_`.
- In `/admin` the company is read from the token (`req.auth.companyId`), never from the URL or body.
- The same routers mounted under `/super-admin/companies/:companyId/…` honour the URL parameter — the super admin may address any tenant.
- Look-ups are scoped by company; a foreign id returns 404, revealing nothing.

6.2 Tenant resolution by domain

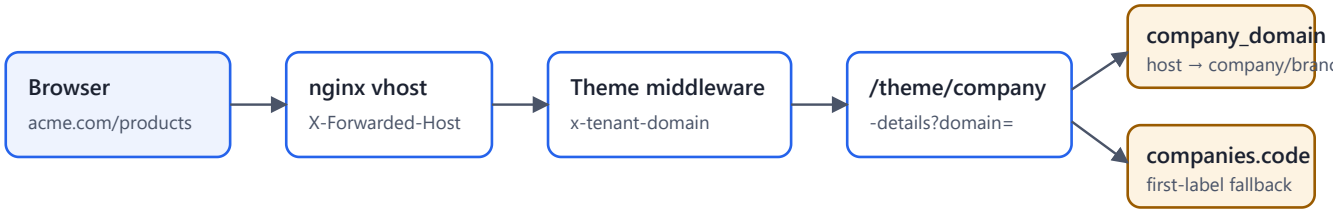


Figure 3 — How a website request finds its tenant. Unknown or inactive hosts receive platform defaults (`resolved:false`), so the endpoint cannot be used to enumerate tenants.

6.3 Branch awareness

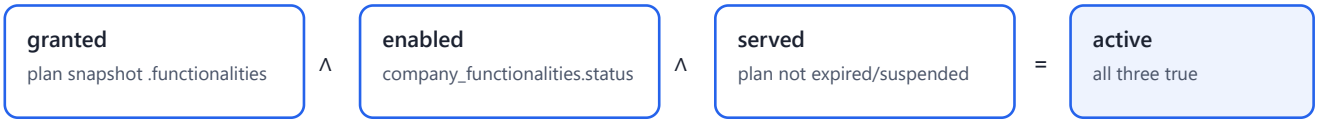
A domain pinned to a branch (`sub_company_id`) serves that branch's logo, favicon, theme colours, slides, About, figures, team, gallery, features and Contact settings, falling back to the company-wide row where the branch has none. Theme colours merge per key: *branch → company → preset theme*.

7. Authentication & authorisation

Scope	Issued by	Token carries	Guards
`super_admin`	`POST /auth/super-admin/login`	id, scope	`authenticate` + `superAdminOnly` on /super-admin
`admin`	`POST /auth/admin/login`	id, scope, companyId	`authenticate` + `adminOnly`, then `requirePermission(slug, action)`
`customer`	`POST /theme/auth/verify` (OTP)	id, scope, companyId	`authenticate` + `customerOnly` on /theme/me

- `authenticate` reloads the principal from the database on every request and rejects inactive/deleted users and inactive companies — a disabled account is locked out immediately, not at token expiry.
- RBAC: `role_permissions` (role × menu × canView/canCreate/canEdit/canDelete/canExport). The main admin (`isCompanyAdmin`) and super admins bypass it.
- `/auth/me` returns the permission map and menu tree; menus are dropped when the plan does not grant their functionality (`MENU_FUNCTIONALITY`), and children in `MENU_INHERITS_PARENT` follow the parent's permission.
- The Admin UI mirrors the API: `permissionGuard` on routes, `*appCan` on buttons — but the API is the enforcement point.

8. Feature entitlement



Writes check granted (assertGranted) so content can be prepared while switched off · Public payload serves only active features (others are null)

Figure 4 — Entitlement is decided in one place: `services/functionality.service.js`

9. Key flows

9.1 Website page render

1. Request reaches the design's container via nginx; middleware sets `x-tenant-domain`.
2. Layout and page call `getCompanyDetails()` (one API call per render thanks to fetch memoisation).
3. Guard chain: API unreachable → ServiceUnavailable; unresolved → CompanyNotFound; `service.active=false` → PlanNotice; feature absent → 404.
4. Sections render from the payload; brand colours become CSS variables on `<body>`.

9.2 Place an order

1. Client cart → Server Action `submitOrder` → `POST /theme/orders` with domain, product ids, quantities, contact, optional address id and Bearer token.
2. `order.service` checks entitlement (products + orders active, route usable), prices every line from the catalogue, snapshots lines and address, assigns `ORD…` number, stores *pending*.
3. Response returns number, total and the WhatsApp text / payment link; the browser opens the hand-off.

9.3 Provision a company

One transaction in `company.service.createCompany`: company → head-office branch (`CODE-HO`) → optional domain → system role + full permissions → three default slides → main admin (generated password returned once) → optional subscription activation + transaction.

9.4 Subscription sweeper

`runDueSubscriptions()` runs at boot, hourly, and on demand: activates due *pending* terms, renews *autoRenew* terms at end, expires terms past `endDate + graceDays`, writing a `subscription_events` row for each move.

10. State machines

Subscription (SUBSCRIPTION_TRANSITIONS)

```
pending   → active | cancelled
active    → suspended | expired | cancelled
| superseded*
suspended → active | expired | cancelled
expired, cancelled, superseded: final
* superseded only through a plan change
```

Order (ORDER_TRANSITIONS)

```
pending → confirmed → packed → dispatched →
delivered
any non-terminal → cancelled
confirmed: reserve stock · dispatched: issue
stock
cancelled: release or return stock
payment: unpaid → paid → refunded (paid →
unpaid allowed)
```

Booking (BOOKING_TRANSITIONS)

```
requested → accepted | declined | cancelled
accepted  → completed | cancelled
requested and accepted hold the slot
```

Invoice

```
draft → issued → cancelled
draft: editable, deletable
issued: stock deducted, payments
unpaid|partial|paid
cancelled: stock returned
```

11. Cross-cutting concerns

Concern	Approach
Response envelope	`{ success, message, data, meta?, errors?: [{field, message}] }` via `utils/response.js`.
Errors	`ApiError.badRequest/unauthorized/forbidden/notFound/conflict/internal`; Sequelize errors mapped (unique → 409, validation → 400, FK → 400).
Pagination	`?page&limit&search&status&sortBy&sortOrder`; limit ≤ 200; sort allow-list.
Transactions	Multi-step writes (provisioning, plan change, order status + stock, invoice issue) run in one Sequelize transaction.
Audit	`createdBy/updatedBy`, subscription events, stock ledger with `balanceAfter`.
Storage	Upload ledger; limit = company override → plan storage → unlimited; enforced on upload.
Rate limiting	Login 30/15 min; theme reads 120/5 min; orders 15/h; service leads 12/h; testimonials 5/h; customer auth 10/15 min; website enquiries 10/15 min.
Time	MySQL timezone +05:30; companies carry their own timezone; analytics series gap-filled server-side.
Logging	morgan access log; `utils/logger` levels; debug off in production.

12. Architecture decisions (ADR summary)

#	Decision	Reason
1	Shared schema with `companyId`, not a database per tenant	Small tenants, many of them; one sync, one backup, one deploy.
2	One deployment per website design, tenant resolved by host	Onboarding without builds; designs upgrade for all tenants at once.
3	Company from token, never from client	Makes cross-tenant access impossible by construction.
4	Snapshots for plans, order lines, addresses	History must not change when sources are edited.
5	Soft delete everywhere (except ledgers/switches)	Recoverability; uniqueness checked in services instead of DB indexes.
6	Stock = ledger + running total in one transaction	Truth and speed; drift traceable via `balanceAfter`.
7	Sequelize alter-sync instead of migrations	Speed of iteration; trade-off — production schema changes need care (see Document 07).
8	No UI/chart libraries in consoles	Small bundles, one design system, hand-written SVG charts starting at zero.
9	Constants file as the single catalogue	Adding a feature or enum is one entry; validators, Swagger and UIs read it.